ACHELION CAPITAL MANAGEMENT, LLC

ARMS Infrastructure Specification

Daily Monitor Layout · Mac Mini Server · Three-Component Persistent Architecture

For: Josh (Lead Developer)

Version: 1.0 — March 25, 2026

Classification: CONFIDENTIAL — GP and Development Use Only

Companion document to ARMS FSD v1.1. Read alongside, not instead of.

"Silence is trust in the architecture."

0. Purpose of This Document

This document defines two things that ARMS FSD v1.1 does not cover in sufficient detail for Josh to build without ambiguity: (1) the exact layout specification for the daily ARMS monitor — every section, its data source, its update frequency, and its position in the operational sequence; and (2) the complete Mac Mini server architecture that gives ARMS a persistent home, a continuous heartbeat, and a permanent intelligence layer.

The monitor layout is not a design preference. It is an operational specification that encodes the PM Playbook v4.0 daily assessment sequence into a visual instrument. Build it in a different order and it stops being a decision tool and becomes a dashboard. Those are not the same thing.

The Mac Mini architecture closes three structural gaps identified in the strategic review: the absence of a persistent knowledge base, the absence of a scheduled autonomous execution layer, and the absence of a direct Claude API integration that makes intelligence available programmatically rather than through a chat interface.

PART ONE — Daily Monitor Layout Specification

1. Design Principle

The monitor is a decision instrument, not a dashboard. Sequence encodes operational logic.

The daily monitor layout mirrors the PM Playbook v4.0 daily assessment sequence exactly. The PM reads top to bottom and every section they need to make a decision is present before the section that requires that decision. Macro context before positions. Module status before alerts. Alerts before the decision queue. The decision queue last.

This sequence is not negotiable. A dashboard that puts the equity book first and the regime assessment second has inverted the logic. A PM who looks at position prices before understanding the regime ceiling they are operating under is flying the jet before checking the instruments. The monitor enforces discipline through layout.

Delivery time	6:00 AM CT daily — automated, no human involvement
Update frequency	Regime score: continuous (every 5 min via RPE). Prices: real-time during market hours. Module status: post-snapshot (5:30 AM CT). Alerts: real-time triggered.
Delivery methods	Primary: web dashboard (local network or secure tunnel). Secondary: PDF email delivery. Both auto-generated from the same data source.
Session log	Every monitor generation event is logged with timestamp. Immutable. LP narrative engine reads from this log.
Override	No human edits the monitor output. It is generated from system data. If data is wrong, fix the data source. Never manually edit the output.

2. Monitor Section Specifications

Each section is specified below with its exact content, data sources, update trigger, and rendering requirements. Josh builds each section as a discrete module that the daily_monitor.py assembles in the defined order.

Section 0 — Header Block

Position: top of monitor. Always visible. Never omitted.

Field	Content	Data source	Update trigger	Render rule
Fund name	Achelion Capital Management, LLC	Static config	Never	Left-aligned, 18px
Monitor title	ARMS Daily Monitor	Static	Never	18px bold
Date + time	Wednesday, March 25, 2026 · 6:00 AM CT	System clock	Each generation	Format: DayName, Month DD, YYYY · HH:MM TZ
Architecture label	Architecture AB v4.0 · 58/20/14/8	Static config	On architecture change only	12px muted
LAEP mode badge	NORMAL / ELEVATED / CRISIS	LAEP module output	VIX threshold cross	Color: green/amber/red. Always shown.
Current regime badge	RISK_ON / WATCH / NEUTRAL / DEFENSIVE / CRASH	ARAS output	Each regime score cycle	Color-coded. Top right. Always shown.

Section 1 — Macro Compass · Regime Scoring

Position: immediately after header. First thing the PM reads. Sets the operational context for everything below.

This section shows the current and previous regime with scores, the transition driver narrative between them, and the regime scoring key. It is the most important section in the monitor. All other sections are contextualised by what is shown here.

Field	Content	Data source	Update trigger	Render rule

Previous regime label	Canonical regime name (e.g., WATCH)	ARAS session log	Each monitor generation	Left column. Badge with regime color.
Previous regime score	Composite score 0.0–1.0	ARAS session log	Each monitor generation	Score bar + numeric label
Previous score inputs	3–5 bullet inputs that drove the score	ARAS signal log	Each generation	11px muted text below bar
Previous ceiling + SLOF	Ceiling % · SLOF active/removed	ARAS session log	Each generation	Right column. Two rows.
Transition driver	1–2 sentence narrative of what changed	AI-generated from signal delta	Each generation	Center separator row. Italic.
Current regime label	Canonical regime name	ARAS current output	Each 5-min RPE cycle	Left column. Badge with regime color.
Current regime score	Composite score 0.0–1.0	ARAS current output	Each 5-min RPE cycle	Score bar. Color: green ≤0.50 · amber 0.51–0.65 · red >0.65
Current score inputs	Live macro inputs driving score	ARAS signal feeds	Each 5-min cycle	11px muted text. Max 6 bullet points.
Current ceiling + SLOF	Ceiling % · SLOF status	ARAS current output	Each cycle	Right column.
RPE forward signal	Highest transition probability + direction	RPE module output	Each 5-min cycle	Green text below scoring key if >30% transition prob
Scoring key	5-regime score ranges	Static	Never	Bottom of section. 11px muted.

CRITICAL RENDERING RULE: The score bar fill color must encode position within the regime, not just the regime label. A score of 0.68 (low end of DEFENSIVE) renders differently from 0.79 (high end of DEFENSIVE). The bar shows exactly where within the regime the portfolio sits — proximity to the boundary is information.

Section 2 — Macro Inputs

Position: after Macro Compass. Four metric cards. Quantifies the regime drivers in numbers.

Field	Content	Data source	Update trigger	Render rule
QQQ price	Current price · day change · YTD change	Market data feed	Real-time (market hours) · 5-min (pre/post)	Metric card. Delta colored green/red.
Brent crude	Current price · day change · key narrative	Market data feed	Real-time	Metric card. Sub-label: max 5 words context.
10Y Treasury yield	Current yield · day change in bps	Market data feed / FRED	Real-time	Metric card. Direction indicator.
VIX	Current level · day change · LAEP mode trigger	CBOE / market feed	Real-time	Metric card. Badge showing LAEP mode threshold proximity.

Section 3 — Equity Book

Position: after macro inputs. Table format. One row per position. Seven columns.

The equity book table shows all 12 current Architecture AB positions plus any SENTINEL-queued positions in a distinct visual style. Columns are ordered to support rapid scanning — name and weight first, price and delta next, context last.

Column	Width	Data source	Update	Render rule
Position (ticker + company name)	16%	Static book config	On book change	Ticker bold 13px. Company name 11px muted below.
Conviction weight (%)	9%	MICS output · Master Engine	On MICS recalculation	Single decimal. If CDF active: show decayed weight.
Current price	10%	Market data feed	Real-time	'~' prefix during pre/post market. Remove during regular hours.
vs. yesterday (delta)	10%	Market data · prev close	Real-time	Green positive · red negative · neutral for <0.5%
52W range	13%	Market data feed	Daily	Low–high format. Omit if not yet available.

Status narrative	25%	AI-generated from news + ARMS signals	Each monitor generation	Max 8 words. Most important current fact about the position.
Flag badges	17%	Module outputs: CDF, FEM, PERM, ALERT, WATCH, QUEUED	Real-time module triggers	See badge spec below. Max 2 badges per position.

Flag badge specification

Badge	Color	Trigger	Meaning
OK	Grey bg · grey text	Default — no flags active	Position within normal parameters
WATCH	Amber bg · amber text	CDF approaching day 45 · price deterioration · thesis softening	Monitoring heightened. No action required yet.
ALERT	Red bg · red text	CDF day 45+ · FEM ALERT · price −40%+ off high with thesis questions	Action required. Review within 24h.
CDF	Purple bg · purple text	CDF decay factor active (0.80x or 0.60x)	Weight automatically decayed. Shown alongside ALERT or WATCH.
PERM?	Green bg · green text	Unrealized gain >30% on position	PERM covered call evaluation triggered. Tier 1 action queued.
QUEUED	Blue bg · blue text	SENTINEL approved · regime gate blocking entry	Position approved for initiation. Waiting for regime clearance.
MICS:N	Teal bg · dark text	Always shown	Model-implied conviction score for the position. e.g., MICS:7

NOTE: Queued positions appear at the bottom of the equity book table in a visually distinct row (lighter background). They show ticker, MICS score, SENTINEL verdict, and trigger condition. They do not show a current weight because they are not yet in the book.

Section 4 — Defensive Sleeve + Cash

Position: after equity book. Two-column layout. Left: defensive. Right: cash and PTRH.

Field	Content	Data source	Update trigger	Render rule

SGOV	Allocation % · NAV status	Market data + NAV feed	Daily	Status: STABLE / WATCH / ALERT
SGOL	Allocation % · gold spot price	Gold spot feed	Real-time	Show gold price in sub-label
DBMF	Allocation % · trend status	NAV feed + oil/rates signal	Daily	Status narrative max 5 words
STRC	Allocation % · yield · NAV status	NAV feed + yield feed	Daily	Show current yield. NAV vs par.
PTRH (QQQ puts)	Size multiplier · DTE status · sizing	Tail hedge module output	Daily after position snapshot	ALERT if any position ≤30 DTE. Show regime-required multiplier.
STRC reserve (cash)	Allocation % · status	Cash account snapshot	Daily	Status: LOCKED / DEPLOY-READY based on regime
Core hedge (cash)	Allocation %	Cash account	Daily	Money market label
Ops buffer (cash)	Allocation %	Cash account	Daily	Money market label

Section 5 — v4.0 Module Status

Position: after defensive sleeve. Six cards in two rows of three. One card per key module.

This section provides a rapid status check on the six most operationally significant modules. Each card shows module name, current status value, and a one-sentence note. Color coding: green = normal, amber = watch/advisory, red = action required.

Module	Status values	Data source	Color rule	Card position
PDS — Drawdown Sentinel	NORMAL / WARNING / DELEVERAGE_1 / DELEVERAGE_2	drawdown_sentinel.py output	Green=NORMAL · Amber=WARNING · Red=DELEVERAGE	Row 1, Card 1
FEM — Factor Exposure	NORMAL / WATCH / ALERT + highest factor name	factor_exposure.py output	Green=NORMAL · Amber=WATCH · Red=ALERT	Row 1, Card 2
CDF — Conviction Decay	No flags / [TICKER]: day N / [TICKER]: DECAYING	perm.py CDF output	Green=clean · Amber=approaching · Red=decaying	Row 1, Card 3

PTRH — Tail Hedge	[multiplier]× base · DTE status	tail_hedge.py output	Green=sized correctly · Red=roll required or wrong size	Row 2, Card 1
LAEP — Execution Mode	NORMAL / ELEVATED / CRISIS	VIX threshold logic	Green=NORMAL · Amber=ELEVATED · Red=CRISIS	Row 2, Card 2
RPE — Regime Probability	Highest transition: [REGIME] [X]% · [N]h	regime_probability.py output	Green if current regime stable · Amber if >30% transition prob	Row 2, Card 3

Section 6 — ARMS Alerts

Position: after module status. Variable number of alert boxes. Ordered by severity: danger → warn → info → ok.

Each alert is a distinct box with a left border color encoding severity, a bold header line, and a body explanation of 2–4 sentences. Alerts are generated automatically by modules. No human writes alerts manually. If no alerts are active, this section shows a single green 'All systems normal' box.

Alert type	Border color	Background	Trigger	Max per session
⚑ DANGER	Red (#D85A30)	Light red (#FAECE7)	PDS DELEVERAGE · CDF day 135 · regime CRASH · kill chain fire	No limit — all must show
⚠ WARNING	Amber (#EF9F27)	Light amber (#FAEEDA)	FEM ALERT · CDF day 45+ · PTRH roll required · regime approaching boundary	No limit
ℹ INFO	Blue (#378ADD)	Light blue (#E6F1FB)	SENTINEL queue update · RPE transition signal · GOOG regime trigger	No limit
✓ OK	Green (#639922)	Light green (#EAF3DE)	Module operating as designed · defensive sleeve performing · no action required	Max 3 — consolidate if more
⚑ PURPLE	Purple (#7F77DD)	Light purple (#EEEDFE)	Major macro development · geopolitical signal · ceasefire · catalyst event	Max 1 per session — top priority

ORDERING RULE: Purple (major macro development) always first if present. Then danger. Then warning. Then info. Then ok. Never reorder within severity tier. The PM reads alerts top to bottom and acts in sequence.

Section 7 — PM Decision Queue

Position: last section. Always last. The PM has read everything above before arriving here.

The decision queue is the only section of the monitor that requires PM action. Each item is a numbered alert box (red/amber/blue) with a bold action header and a 1–2 sentence instruction. Items are ordered by urgency. Tier 0 actions never appear here — they have already executed. Tier 1 items appear here with their veto window noted. Tier 2 and 3 items appear here with their confirmation requirement noted.

Maximum items	7. If more than 7 actions are queued, the system has failed to execute Tier 0 actions correctly. Investigate.
Item numbering	Sequential 1–N. Numbered bullets. Never unnumbered.
Item format	[Number]. [BOLD ACTION LABEL]: [1-2 sentence instruction with specific data points.]
Carryover items	If an item appeared in yesterday's queue and was not actioned, it shows in red with '⚑ CARRIED OVER FROM [DATE]' prefix. Two carryovers = GP notification triggered.
Tier 1 items	Show veto window remaining. e.g., 'Veto window: 2h 14m remaining. No response = execute.'
Completion	Items disappear from queue when actioned or when Tier 1 window expires and system executes. Never manually removed.

Section 8 — Footer

Footer is fixed at the bottom of every monitor. Contains: data source attribution, generation timestamp, current regime score (numeric), previous regime score (numeric), Architecture AB version label, fund name, confidentiality notice. Auto-generated. No human edits.

Example footer: 'Data sourced from public markets · Wednesday, March 25, 2026 · 6:00 AM CT · Regime score: DEFENSIVE 0.68 (prev: DEFENSIVE 0.74) · Architecture AB v4.0 · Achelion Capital Management, LLC · Not for distribution'

3. Rendering Implementation

The monitor is rendered in two formats from a single data source. Josh builds the data assembly layer once. The rendering layer is separate and produces both outputs.

Web dashboard	Flask or FastAPI serving an HTML template. Accessible on local network via Mac Mini IP address or via Cloudflare Tunnel for remote access. Renders in any browser. Auto-refreshes every 5 minutes during market hours. Static at 6AM CT generation pre-market.
PDF email delivery	Playwright or WeasyPrint converts the HTML monitor to PDF. SendGrid free tier (100 emails/day) or SMTP via Gmail delivers to PM and GPs at 6:00 AM CT. PDF is the LP-ready format — same content, exportable.
CSS design system	Section colors, badge colors, alert box colors, and typography must exactly match the specification in this document. No creative interpretation. The layout encodes operational logic — visual deviation is a specification error.
Dark mode	Monitor must render correctly in both light and dark browser modes. Use CSS variables for all colors. Never hardcode hex values in the HTML template.
Mobile responsive	The monitor must be readable on a phone without horizontal scrolling. Use responsive grid for the four macro metric cards and the two-column defensive sleeve section.

PART TWO

Mac Mini Server Architecture

4. Hardware Specification

The Mac Mini M4 Pro is the correct choice. Institutional quality at pre-institutional cost.

The Mac Mini is not a compromise. For a pre-launch fund running ARMS at current AUM, it is the correct infrastructure choice. Cloud hosting (AWS/GCP) costs more monthly, introduces external dependency, and adds latency between ARMS modules. A Mac Mini in a secure location with proper networking is more reliable, more private, and more cost-effective for Phase 1 through Phase 3.

Component	Specification	Rationale
Model	Mac Mini M4 Pro	M4 Pro has 14-core CPU, 20-core GPU, 24GB unified RAM minimum. Runs Python ARMS stack, PostgreSQL, Chroma, Flask, and Prefect simultaneously without throttling.
RAM	24GB minimum — 48GB preferred	Chroma vector DB, PostgreSQL, and multiple Python processes running concurrently. 24GB is the floor. 48GB gives headroom for Phase 3 ML workloads.
Storage	512GB SSD minimum — 1TB preferred	PostgreSQL session log, Chroma vector embeddings, ARMS codebase, and 2 years of rolling data fit comfortably in 512GB. 1TB recommended for Phase 3 growth.
Network	Gigabit Ethernet — wired, not WiFi	ARMS data ingestion and API calls require consistent low-latency connectivity. Never run ARMS on WiFi. Static IP on local network.
UPS (Uninterruptible Power Supply)	APC Back-UPS 600VA minimum (~$80)	PostgreSQL and Chroma require clean shutdown to prevent database corruption. The UPS provides 10-20 minutes of runtime on power loss — enough to execute a graceful shutdown script.

Backup	External SSD 1TB + Time Machine enabled	Daily encrypted backup of the entire ARMS data directory. PostgreSQL pg_dump scheduled nightly. Chroma snapshot scheduled nightly. Off-site backup via encrypted cloud (Backblaze B2, ~$7/month).
Location	Secure, climate-controlled, always-on	Not a home office desk that gets powered off. A dedicated location — office server rack shelf, closet with ventilation, or colocation cage. Mac Mini is fanless under normal load — runs cool and silent.
Approximate total cost	$1,400–$1,800 hardware + ~$80 UPS + $7/month backup	One-time capital cost. No monthly cloud fees for compute. Payback vs. equivalent AWS EC2 instance: 8-12 months.

5. Three-Component Architecture

The Mac Mini closes three structural gaps identified in the strategic review. Each gap has a component. Each component has a specific technology stack, a build specification, and a 'mile later' upgrade path. Build all three in Phase 1 — they are not optional and they are not expensive.

Component 1 — Persistent Knowledge Base

Gap closed: ARMS currently has no memory between sessions. Context resets. Knowledge does not compound.

A vector database stores every ARMS document, SENTINEL analysis, session log entry, and monitor output as searchable embeddings. When any intelligence task is run — SENTINEL analysis, LP digest generation, scan engine operation — the relevant context is retrieved from the knowledge base and injected into the API call. The knowledge base compounds over time. The longer ARMS runs, the richer the context it can provide.

Technology	Chroma DB (open source, free, runs locally on Mac Mini)
Installation	pip install chromadb — one command
Storage location	~/achelion/knowledge_base/ on Mac Mini SSD
Collections	5 collections: arms_documents, sentinel_analyses, session_log, monitor_outputs, thesis_seeds
Embedding model	OpenAI text-embedding-3-small via API ($0.02/1M tokens — negligible cost) OR sentence-transformers locally (free, slightly lower quality)
Retrieval	Top-K semantic search. Each Claude API call retrieves the 5 most relevant chunks from the knowledge base before generating output.
Update frequency	arms_documents: on document change. sentinel_analyses: after each SENTINEL run. session_log: real-time append. monitor_outputs: after each generation. thesis_seeds: on PM submission.
Backup	Chroma snapshot nightly to external SSD + Backblaze B2

Knowledge base population — day one

1. Run the ingestion script on all existing ARMS documents: PM Playbook v4.0, THB v4.0, FSD v1.1, this document, the full-cycle backtest, all SENTINEL analyses run to date.
2. Ingest all session log entries from the first day ARMS begins logging. The log is the primary training data for the system's self-improvement loop.

3. Set up the nightly incremental ingestion job via Prefect. New session log entries, monitor outputs, and SENTINEL analyses are embedded and stored automatically.

How it connects to Claude API calls

```
# Every intelligence task follows this pattern:
def run_intelligence_task(query: str, task_type: str) -> str:
    # 1. Retrieve relevant context from knowledge base
    context = knowledge_base.query(
        collection=task_type,
        query_text=query,
        n_results=5
    )
    # 2. Inject context into Claude API call
    response = claude_api.call(
        system=ARMS_SYSTEM_PROMPT,
        context=context,
        query=query
    )
    # 3. Store response back into knowledge base
    knowledge_base.add(collection=task_type, text=response)
    return response
```

Component 2 — Scheduled Autonomous Execution Layer

Gap closed: ARMS modules currently produce signals. Nothing acts on them automatically. This closes that gap.

The scheduler is the heartbeat of the system. It triggers every ARMS module at its defined cadence, passes outputs to downstream modules, and ensures the entire pipeline runs without human involvement. When the scheduler runs correctly, the PM wakes up to a monitor that has already assessed the regime, flagged positions, computed scenario P&L, and queued decision items — all before 6AM CT.

Technology	Prefect (open source, free tier, runs locally on Mac Mini)
Installation	pip install prefect — one command. prefect server start to run locally.
Alternative	APScheduler (simpler, no UI) if Prefect overhead is unwanted for Phase 1
Storage location	~/achelion/scheduler/ on Mac Mini
Failure handling	Every scheduled job has a retry policy (3 attempts, 60-second backoff). Failures alert to PM via email. Session log records all failures with stack trace.

Master schedule — all ARMS jobs

Job	Cadence	Module	Output
Market data ingestion	Every 5 min (market hours)	market_data.py	SignalRecord list → ARAS, RPE
Regime probability update	Every 5 min	regime_probability.py	RegimeProbabilitySignal → ARAS advisory
ARAS composite score	Every 5 min	aras.py	Regime label + ceiling → Master Engine
Portfolio snapshot	5:15 AM CT daily	master_engine.py	Current weights + MICS scores → all modules
FEM daily scan	5:20 AM CT daily	factor_exposure.py	FactorExposureSignal → daily monitor
SSL scenario computation	5:25 AM CT daily	stress_scenarios.py	ScenarioResults → daily monitor
PDS NAV check	5:25 AM CT daily	drawdown_sentinel.py	DrawdownSentinelSignal → daily monitor
CDF day count update	5:25 AM CT daily	perm.py (CDF)	Decay factors updated → Master Engine
Tail hedge DTE check	5:25 AM CT daily	tail_hedge.py	TailHedgeStatus → daily monitor alert if ≤30 DTE
Daily monitor generation	5:30 AM CT daily	daily_monitor.py	HTML + PDF monitor → delivery pipeline
Monitor delivery (email + web)	6:00 AM CT daily	delivery.py	PDF email · web dashboard refresh
Systematic Scan Engine	Every Monday 5:00 AM CT	systematic_scan.py	SENTINEL candidates → Monday monitor Section 3
Knowledge base incremental ingest	Nightly 2:00 AM CT	kb_ingest.py	New embeddings → Chroma DB
PostgreSQL backup (pg_dump)	Nightly 2:30 AM CT	backup.py	Compressed dump → external SSD + Backblaze
Session Log Analytics	1st of each month 6:00 AM CT	session_log_analytics.py	3 learning loop metrics → monthly monitor section
Proactive Intelligence Digest	1st of each month 7:00 AM CT	proactive_digest.py (Phase 3)	PDF LP report → email delivery

Component 3 — Claude API Integration Layer

Gap closed: intelligence tasks currently require a human chat interface. This makes them programmatic, continuous, and auditable.

The Claude API integration layer is a single Python module that every ARMS intelligence task routes through. It manages the system prompt, context injection from the knowledge base, API call formatting, response parsing, error handling, cost tracking, and result storage. No ARMS module calls the Claude API directly — they all call this wrapper.

Technology	Anthropic Python SDK (pip install anthropic)
Model	claude-sonnet-4-6 — optimal balance of intelligence and cost for ARMS tasks
File	src/intelligence/claude_wrapper.py
System prompt	Loaded from ~/achelion/config/arms_system_prompt.txt — contains full ARMS context, regime definitions, SENTINEL gate criteria, Architecture AB spec. Updated when architecture changes.
Cost tracking	Every API call logs: tokens in, tokens out, estimated cost, task type, timestamp. Monthly cost report generated automatically. Current target: <$300/month total API spend.
Rate limiting	Built-in retry with exponential backoff. Max 3 retries. Failures logged and PM notified.
Response validation	Every response validated against expected schema before use. JSON responses parsed and validated. Malformed responses logged and flagged.

Intelligence tasks routed through the wrapper

Task	Trigger	Approx. cost/call	Output
SENTINEL Gate 1+2 analysis	PM thesis seed submission	~$0.40	JSON: {gate1_pass, gate2_pass, rationales}
SENTINEL Gate 3 mispricing score	Gates 1+2 pass	~$0.20	JSON: {g3_score, component_scores}
SENTINEL adversarial cross-examination	Gates 1-3 pass	~$0.30	JSON: {bear_evidence_present, bear_rationale}
Systematic Scan Engine — per company	Every Monday, per company in universe	~$0.40	JSON: {gate1_pass, gate2_pass, g3_estimate}
Regime transition narrative	Each monitor generation	~$0.10	String: 1-2 sentence narrative

Position status narrative	Each monitor generation, per position	~$0.05	String: max 8 words
Alert generation	Module flag triggered	~$0.10	String: bold header + 2-4 sentence body
LP narrative on PDS DELEVERAGE_2	PDS module trigger	~$0.50	String: full disclosure narrative
Proactive Intelligence Digest	Monthly (Phase 3)	~$1.00	Full digest document
Thesis Retirement analysis	TRP 3-condition trigger	~$0.40	JSON: {retire_recommendation, rationale}

ARMS system prompt — what Claude knows at every call

The system prompt loaded at every API call contains the following in structured sections. This is what makes every API response ARMS-aware rather than generic:

4. Complete Architecture AB specification: 58/20/14/8 · current equity book · conviction weights · defensive sleeve composition.
5. Five canonical regime labels and their ceilings, SLOF status, and behavioral rules.
6. SENTINEL gate criteria verbatim from PM Playbook v4.0.
7. MICS formula and source quality taxonomy.
8. Current regime label and score (injected fresh at each call).
9. Top 3 most recent session log entries (injected from knowledge base retrieval).
10. Current FEM status and active flags (injected from module output).

6. Complete Mac Mini Software Stack

Layer	Technology	Cost	Purpose
OS	macOS Sequoia (built-in)	$0	Base OS. Unix-compatible. Python, PostgreSQL, all deps run natively.
Python runtime	Python 3.11 via Homebrew	$0	ARMS codebase runtime.
Relational DB	PostgreSQL 16 via Homebrew	$0	Session log, portfolio snapshots, order history, audit trail.
Vector DB	Chroma DB (pip install)	$0	Persistent knowledge base. Semantic retrieval for Claude API calls.
Scheduler	Prefect (pip install, local server)	$0	All ARMS job scheduling. Retry logic. Failure alerting.
Web framework	FastAPI + Uvicorn (pip install)	$0	Daily monitor web dashboard. ARMS API endpoints.
PDF generation	WeasyPrint (pip install)	$0	HTML→PDF for monitor email delivery and LP reports.
Email delivery	SendGrid free tier OR Gmail SMTP	$0	Monitor delivery at 6AM CT. LP narrative delivery.
Remote access	Cloudflare Tunnel (free)	$0	Secure HTTPS access to web dashboard from anywhere. No port forwarding. No VPN required.
Broker API	ib_insync (pip install) for IBKR	$0	Order submission, position snapshot, fill confirmation.
AI intelligence layer	Anthropic Python SDK (pip install)	~$150-300/month	All ARMS intelligence tasks. SENTINEL, scan engine, narratives.
Embeddings	OpenAI text-embedding-3-small OR sentence-transformers	~$5/month OR $0	Knowledge base document embedding.
Cloud backup	Backblaze B2	~$7/month	Off-site encrypted backup of all ARMS data.
Monitoring	Uptime Kuma (Docker, free)	$0	Mac Mini uptime monitoring. Alerts if any service goes down.
Version control	Git + GitHub private repo	$0	All ARMS code versioned. Never lose work.
TOTAL MONTHLY COST	Beyond hardware	~$160-310/month	Fully operational ARMS FSD platform.

7. Network and Security Architecture

Security is not optional for a fund management system. Every component that touches portfolio data, session logs, or LP information must be encrypted at rest and in transit. The following spec is the minimum acceptable standard for a registered investment vehicle.

Security control	Implementation	Rationale
Disk encryption	macOS FileVault enabled on Mac Mini	All ARMS data, session logs, and API keys encrypted at rest. Required for any system holding portfolio data.
API key management	macOS Keychain or HashiCorp Vault (free)	Never hardcode API keys in source code. Load from secure key store at runtime. Rotate keys quarterly.
Network access	Wired ethernet. Router firewall. No public-facing ports except via Cloudflare Tunnel.	Mac Mini never directly exposed to the internet. Cloudflare Tunnel creates an outbound-only encrypted connection.
Web dashboard auth	Cloudflare Access (free tier) — email OTP or Google OAuth	Dashboard is not publicly accessible. PM and GPs authenticate before viewing.
Database encryption	PostgreSQL with encrypted tablespace OR pgcrypto for sensitive columns	Session log and portfolio data encrypted at the column level for the most sensitive fields.
Backup encryption	Restic (free) with AES-256 encryption before Backblaze upload	Backblaze never sees unencrypted data. Key stored in macOS Keychain.
Code repository	Private GitHub repo with 2FA required for all contributors	ARMS codebase never in a public repository. Branch protection on main.
Incident response	UPS graceful shutdown script. Prefect failure alerts to PM email within 2 minutes of any job failure.	System failure does not mean data loss or uncontrolled execution.

8. Mac Mini Setup Sequence — Josh's Day One

Complete in this order. Do not skip steps. Each step is a dependency for the next.

Day 1 — Hardware and Base OS

11. Unbox Mac Mini M4 Pro. Connect via ethernet — not WiFi. Connect to UPS.
12. Enable FileVault disk encryption immediately. Before installing anything.
13. Create a dedicated 'achelion' user account. Do not use as primary personal machine.
14. Install Homebrew: /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
15. brew install python@3.11 postgresql@16 git
16. Configure PostgreSQL to start on login: brew services start postgresql@16
17. Create the ARMS database: createdb achelion_arms
18. Install Uptime Kuma via Docker for system monitoring.

Day 2 — Python Environment and Core Dependencies

19. python3.11 -m venv ~/achelion/venv && source ~/achelion/venv/bin/activate
20. pip install anthropic chromadb prefect fastapi uvicorn weasyprint ib_insync psycopg2-binary openai sentence-transformers restic
21. Clone the ARMS private GitHub repo: git clone [repo URL] ~/achelion/arms
22. Set up macOS Keychain entries for: ANTHROPIC_API_KEY, OPENAI_API_KEY (for embeddings), IBKR credentials, SendGrid API key.
23. Initialize Chroma DB: python -c "import chromadb; client = chromadb.PersistentClient(path='~/achelion/knowledge_base')"
24. Run the knowledge base ingestion script on all existing ARMS documents (FSD v1.1, PM Playbook v4.0, THB v4.0, this document, full-cycle backtest).

Day 3 — Scheduling and Web Dashboard

25. prefect server start — start the local Prefect server.
26. Configure all Prefect flows from the master schedule in Section 5.
27. Build the FastAPI daily monitor web app: uvicorn arms.dashboard:app --host 0.0.0.0 --port 8080
28. Install and configure Cloudflare Tunnel: cloudflared tunnel create achelion — generates a secure public URL for the dashboard.
29. Configure Cloudflare Access: restrict dashboard to PM and GP email addresses via Google OAuth.
30. Test the full pipeline: trigger the daily monitor job manually and verify HTML and PDF output.

Day 4 — Broker API and IBKR Paper Connection

31. Install IBKR Trader Workstation (TWS) or IB Gateway on the Mac Mini.
32. Configure TWS for API access: Enable ActiveX and Socket Clients in settings.
33. Test ib_insync connection: python -c "from ib_insync import *; ib = IB(); ib.connect('127.0.0.1', 7497, clientId=1); print(ib.positions())"
34. Build broker_api.py — the sole interface between ARMS and IBKR. No other module touches IBKR directly.
35. Run the regime transition test from FSD v1.1 Section 8.4 against the paper account.

Day 5 — MICS and Intelligence Layer

36. Build mics.py using the formula in FSD v1.1 Section 11.1. Run it against all existing Architecture AB positions. Compare output to current PM-assigned C-levels. Document any divergence >2 levels for GP review.
37. Build claude_wrapper.py — the Claude API integration layer from Section 5 Component 3.
38. Load the ARMS system prompt from ~/achelion/config/arms_system_prompt.txt.
39. Test a SENTINEL Gate 1+2 analysis call with a test ticker. Verify JSON output matches schema.
40. Run the first Systematic Scan Engine job manually on a 10-company test universe. Verify output and cost.

After Day 5: ARMS has a home, a heartbeat, a brain, and a broker connection. Phase 1 is operational. The PM receives the daily monitor at 6AM CT. The system runs without human involvement. The inches have been taken.

9. On SuperAGI and Manus — Where They Fit

SuperAGI and Manus are general-purpose autonomous agent frameworks. They are powerful and genuinely useful for certain workflows. They are not the right tool for ARMS execution. The distinction matters.

Dimension	General-purpose agent (SuperAGI / Manus)	ARMS architecture
Design goal	Flexibility — figure out how to do something novel	Discipline — execute a defined protocol exactly the same way every time
Decision model	Agent decides what to do next based on context	Module fires when trigger condition is met. No discretion.
Auditability	Variable — depends on agent framework logging	Every action logged with triggering module, signal, and timestamp. Immutable.
Predictability	Lower — agent may choose unexpected paths	Complete — output of every module is deterministic given the same inputs
LP defensibility	Difficult — 'the agent decided' is not an acceptable answer	Complete — every decision traceable to a specific module and trigger condition
Best use in Achelion stack	Intelligence layer above ARMS — news monitoring, transcript synthesis, scan engine analysis, digest drafting	Execution layer — regime enforcement, order submission, position sizing, hedge management

SuperAGI and Manus feed ARMS. They do not run ARMS. The architecture is clear: agent intelligence above, disciplined execution below.

A SuperAGI agent connected to the Mac Mini and the Claude API integration layer could continuously monitor news feeds, summarize earnings transcripts for the Systematic Scan Engine, surface breaking developments for the daily alert section, and draft LP communication. This is a valuable Phase 3 or Phase 4 addition. It plugs into the feed-agnostic data ingestion layer Josh builds in Phase 1 as a plugin — no rearchitecting required.

What it never does: submit orders, change regime ceilings, modify position weights, or override any ARMS module output. Those domains belong to the deterministic execution layer. Any agent that can modify execution is a liability, not an asset, in a regulated fund context.

10. Closing Statement

A Mac Mini, $160/month, and five days of Josh's time. That is the cost of giving ARMS a permanent home.

After Day 5, the following is true: ARMS runs continuously without human involvement. The PM receives a fully specified daily monitor at 6AM CT. Every regime transition executes automatically. Every position flag generates an alert. The knowledge base compounds with every session. The Claude API provides intelligence on demand. The broker connection is ready for live execution the moment paper trading validation is complete.

This is not a prototype. This is the foundation of a fund management platform that has never existed at this size. The infrastructure cost is lower than one month of Bloomberg terminal fees. The capability it unlocks is institutional.

The monitor layout specification in Part One ensures that what the PM reads every morning is not a dashboard — it is a decision instrument that encodes 28 years of risk management discipline into a visual sequence. The Mac Mini architecture in Part Two ensures that instrument is always ready, always current, and always honest.

"Silence is trust in the architecture."

Achelion Capital Management, LLC · Flow. Illumination. Discipline. Conviction.